

iOS组件化避坑心得

星的天空 Lv2

2022年01月24日 22:37 · 阅读 4511

[关注](#)

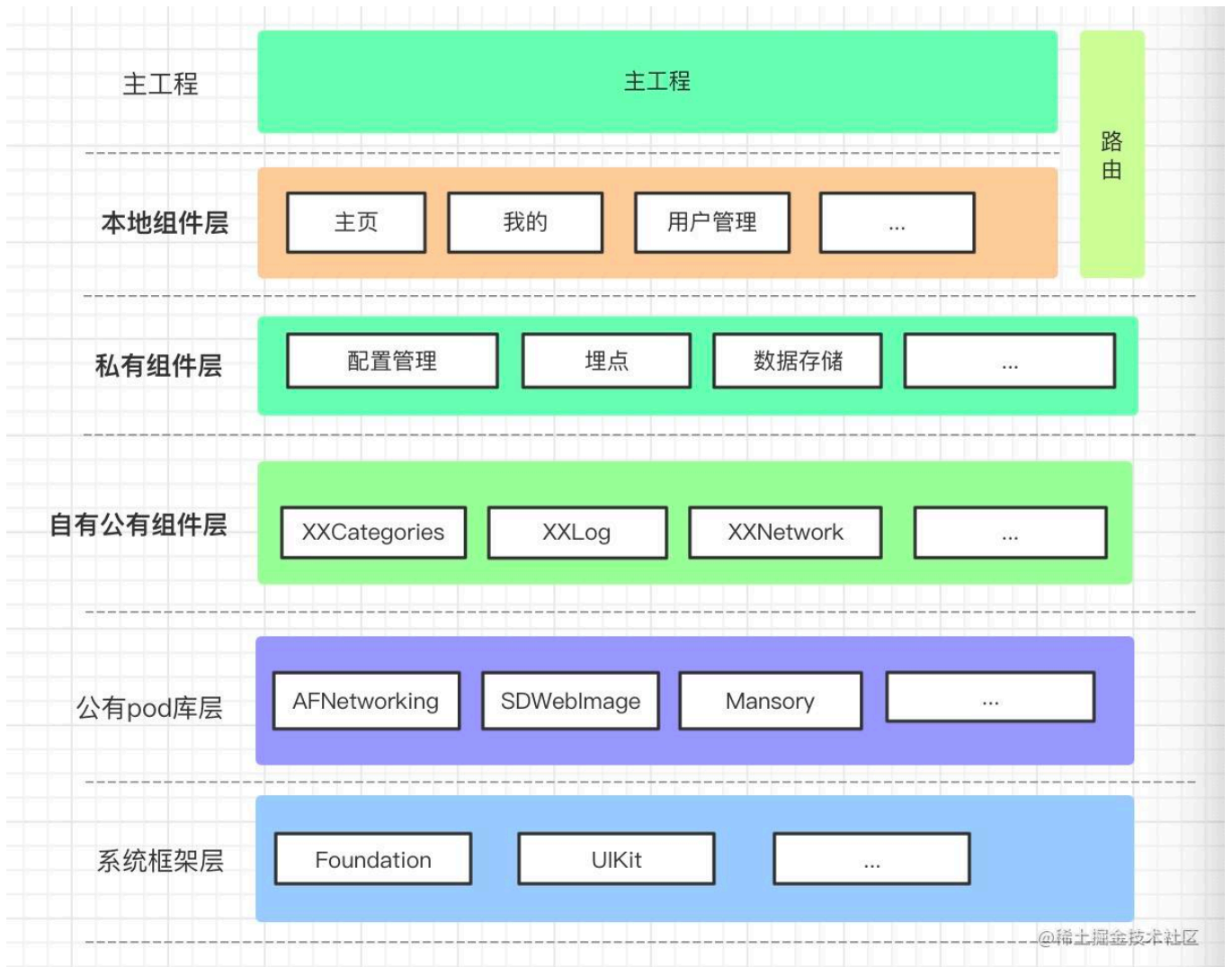
「这是我参与2022首次更文挑战的第1天，活动详情查看：[2022首次更文挑战](#)」。

为什么要组件化

组件化一般是把工程分层拆成不同的组件，以达到解耦，模块复用，便于单元测试，编译速度优化等效果，最终目的是为了提高开发质量和效率。当然，组件化是有一定成本的，在组件化之前要考虑清楚当前的项目情况是否适合组件化，收益能否覆盖开发成本。规模较小，模块没太多复用需求的项目，就没必要进行组件化。

组件化如何分层

组件化之前，先要对项目进行分层，以我们现在的项目为例：



分层后，一般使用 **Cocoapods** 来封装组件，通过 **路由** 来进行不同业务模块间的解耦调用。其中 **系统框架层**，**公有pod库层** 是本来就有的。其它层的拆分逻辑分别是：

- 自有公有组件层（**公有pod**）：这一层的pod都是推到公有仓库，和业务是没有任何耦合的，就是用于放开源的组件。
- 私有组件层（**私有pod**）：这一层的组件都有自己的私有git仓库，通过私有git仓库和私有索引库来进行管理，根据实际业务场景，这一层可能还会再进行分层。一般放一些项目中比较基础和通用的逻辑，例如配置信息管理，埋点管理，数据存储这些。
- 本地组件层（**本地pod**）：这一层放业务模块，是app呈现给用户的一个模块的整体封装，例如我的模块，首页模块这类。它的修改会非常频繁，日常的业务开发大部分都是在这一层，所以使用本地pod的方式，不会单独创建git仓库，与主工程共用一个git仓库进行管理。

`cocoapods` 只支持在 `Podfile` 中以 `path` 方式依赖 `pod`，在 `podspec` 中是不支持的，所以 `本地pod` 最多只能有一层。（你也可以写插件支持多层）

这样分层后，当有新的业务需求时，我们只需要创建一个本地pod，写入pod依赖，就可以快速的进入业务开发状态，因为没有其它模块的干扰，编译和调试的速度会得到极大的提升，同时也避免了模块之间的耦合。

使用 `Cocoapods` 制作不同类型的组件

通过上述内容，可以知道有三种类型的组件：`公有pod`，`私有pod`，`本地pod`。不管什么类型，都建议使用 `pod lib create PodName` 命令来创建组件，在它生成的组件模版基础上，可以很方便的进行开发。

公有Pod

公有pod是面向所有开发者的，需要尽量保证它的可用性和稳定性。如何发布公有pod这里不做说明，可以参考官方文档：[Getting setup with Trunk](#)。一些公司提供的商用公有pod，连pod验证命令 `pod lib lint` 都无法通过，这个是不应该的，最容易出现的问题就是编译错误，因为它会校验各个场景，而很多pod开发者却只保证自己的pod在常见场景可用。常见的报错有：

复制代码

```
// 报错1:
ld: symbol(s) not found for architecture i386

// 报错2:
building for iOS Simulator, but linking in object file built for iOS, file 'you path'
```

这里的 `i386` 是 `32位模拟器` 的架构，`arm64` 是 `m1机型` 上的模拟器架构，出现这种问题一般是pod中引入了 `lib` 或者 `framework` 静态库，而这些库不支持这些架构。现在都 `2022` 年了，微信的最低支持版本都到了 `12.0`，所以请大胆将pod库的 `deployment_target` 参数设置为 `12.0`，这样就不会进行 `i386` 架构的验证。而对

arm64 模拟器架构的支持，最好是将静态库重新打包成 **.xcframework** 格式并支持 **arm64** 模拟器架构，不然会导致 **m1** 设备的使用者只能以 **Rosetta** 模式在模拟器上运行项目。具体可以看我之前写的博文[M1设备的Xcode编译问题深究](#)，如这些静态库是第三方提供的，无法重新打包支持 **arm64** 模拟器架构，则可以进行如下设置避免报错：

```
ruby 复制代码
# 因为依赖的静态库不支持模拟器arm64架构，设置当前这个pod不支持arm64， 以避免pod lib lint无法通过
s.pod_target_xcconfig = { 'EXCLUDED_ARCHS[sdk=iphonesimulator*]' => 'arm64' }
# 单纯设置pod_target_xcconfig只是设置当前这个pod不支持arm64， 这里把这些pod的上层设置为不支持arm64
s.user_target_xcconfig = { 'EXCLUDED_ARCHS[sdk=iphonesimulator*]' => 'arm64' }
```

对于 **公有pod**，负责任的做法是没有任何错误和警告后再进行发布，如果由于各种客观原因，实在无法去除警告，可以加入 **--allow-warnings** 参数来推送：**pod trunk push [NAME.podspec] --allow-warnings**。如果添加 **--skip-import-validation** 参数来逃避验证，则显得有些不负责任了。

另外，在 **CocoaPods** 的 **1.8** 版本，将默认的 **spec repo** 设为了 **CDN源**，以提高pod的速度。刚发布的 **公有pod** 版本，可能要几个小时后才能被同步到 **CDN源**，导致刚发布时调用 **pod install --repo-update** 没法找到新发布的pod库，这时可以通过指定源来解决这个问题，样例如下：

```
pod '你的公有pod', :source => 'https://github.com/CocoaPods/Specs'
```

复制代码

当然，**CDN源** 同步后要记得改回。

私有Pod

私有pod 一般通过 **私有repo** 来进行管理，这样才方便做版本管理和使用缓存。**私有repo** 创建命令是 **pod repo add REPO_NAME SOURCE_URL**，其中 **SOURCE_URL** 就是 **私有repo** 的git地址。创建 **私有repo** 后，通过 **pod repo push REPO_NAME SPEC_NAME.podspec** 命令来发布 **私有pod** 到 **私有repo**。需要注意的是 **私有pod** 和 **公有pod** 的发布命令并不一样，分别是：

[ruby](#) 复制代码

```
# 公有pod发布命令
pod trunk push SPEC_NAME.podspec
# 私有pod发布命令
pod repo push REPO_NAME SPEC_NAME.podspec
```

发布后，需要在 **Podfile** 中加入 **私有repo源**，才能找到私有pod并安装成功。

[ruby](#) 复制代码

```
# --- Podfile文件 ---
# 指定私有source
source 'https://youhost.com/YouPrivateRepo.git'
# 指定公有source,
source 'https://github.com/CocoaPods/Specs.git'
```

这些固定流程不做过多说明，具体内容可以查看官方文档:[Private Pods](#)

如果私有pod中依赖了非公有源的pod，在 **pod lib lint** 时会出现这类报错：

[复制代码](#)

```
ERROR | [iOS] unknown: Encountered an unknown error (Unable to find a specification fo
```

这时根据提示调用 **pod repo update** 后也是无效的，这类问题可以通过指定 **sources** 来解决，以如上报错为例，它依赖的 **AlicloudHTTPDNS** 是阿里的源：**https://github.com/aliyun/aliyun-specs.git**，设置 **sources** 后则可以验证通过：

[ruby](#) 复制代码

```
# 可以通过逗号分隔，添加多个不同的source。(--sources='a_source,b_source')
pod lib lint DTHttpDns.podspec --allow-warnings --sources='https://github.com/aliyun/a
```

pod repo push 也可能会出现上述报错，但是它的逻辑稍有不同，它会从你本地的repo列表中去查找pod依赖，找到则不会报错。而 **pod lib lint** 在没有指定 **sources** 时，只会从默认的源去找。

本地Pod

本地pod 与主工程一起被同一个git仓库管理，不需要单独进行版本管理，也不需要 push，而是在 Podfile 中直接以 path 的方式进行引入，样例如下：

```
# --- Podfile文件中 ---
pod '你的本地pod', :path => '../本地pod路径/你的本地pod目录名'
```

[ruby 复制代码](#)

Pod组件中使用资源的坑

在pod中，经常会出现需要使用图片，xib，json文件等资源的场景，建议使用 resource_bundles 来配置使用这些资源，以名为 DTVideo 的pod库为例：

```
# -- DTVideo.podspec 文件中--
s.resource_bundles = {
  'DTVideoAssets' => ['DTVideo/{Assets,Classes}/**/*.{xib,xcassets}']
}
```

[ruby 复制代码](#)

这样配置后， cocoapods 会自动把这些资源打包成一个名叫 DTVideoAssets 的 bundle 文件，在pod中使用这些资源的方式会发生一些改变。假如这个pod中有一个类 VideoPlayListCell.swift，那么我们可以创建辅助方法：

```
struct DTVideoCommon {
    static func assetsBundle() -> Bundle? {
        let myBundle = Bundle(for: VideoPlayListCell.self)
        let path = myBundle.path(forResource: "DTVideoAssets", ofType: "bundle")
        guard let path = path else {
            return nil
        }
        let assetsBundle = Bundle.init(path: path)
        return assetsBundle
    }

    static func imageWith(named name: String) -> UIImage? {
        let assetsBundle = assetsBundle()
    }
}
```

[swift 复制代码](#)

```
        let image = UIImage.init(named: name, in: assetsBundle, compatibleWith: nil)
        return image
    }
}
```

[复制代码](#)

加载图片时：

```
let image = DTVideoCommon.imageWith(named: "video_play_max_nor")
imageView.image = image
```

[swift 复制代码](#)

使用xib时：

```
let cellNib = UINib.init(nibName: cellIdentifier, bundle: DTVideoCommon.assetsBundle())
tableView.register(cellNib, forCellReuseIdentifier: cellIdentifier)
```

[swift 复制代码](#)

xib文件中设置Module名的坑

xib 文件中有 Module 设置，如果是在工程中创建的，那么它默认是勾选上 inherit Module From Target，当将这个文件移动到pod中时，它的 Module 名就被设置成了默认名，即 bundle 名，这样会导致创建这个cell的时候报错：

[复制代码](#)

```
Terminating app due to uncaught exception 'NSUnknownKeyException', reason: '[<UITableView
```

错误设置图示如下：



正确的设置是不要勾选 `inherit Module From Target`，并且在Module栏输入正确的 `Module名`。

当然你也可以空着，不输入 `Module` 名，但是这样需要修改 `VideoPlayListCell` 的类名：

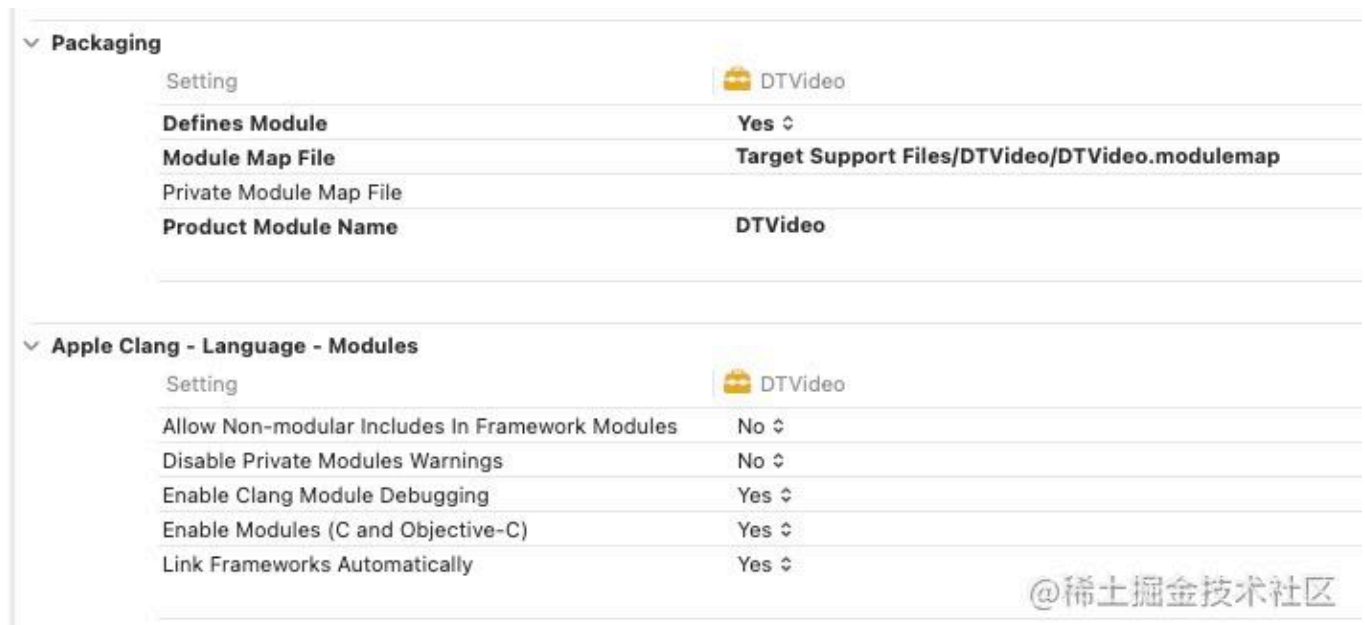
```
@objc(VideoPlayListCell) // Module栏空着情况下，必须添加这行
class VideoPlayListCell: UITableViewCell {
    // ... 代码省略
}
```

[swift 复制代码](#)

如果不使用 `@objc(VideoPlayListCell)` 修改类名，那么会出现上面一样的报错，所以还是建议输入正确的 `Module名`

关于module

`cocoapods` 现在默认会开启pod库的 `module` 选项，所以没有必要在 `podspec` 中配置：`'DEFINES_MODULE' => 'YES'`，



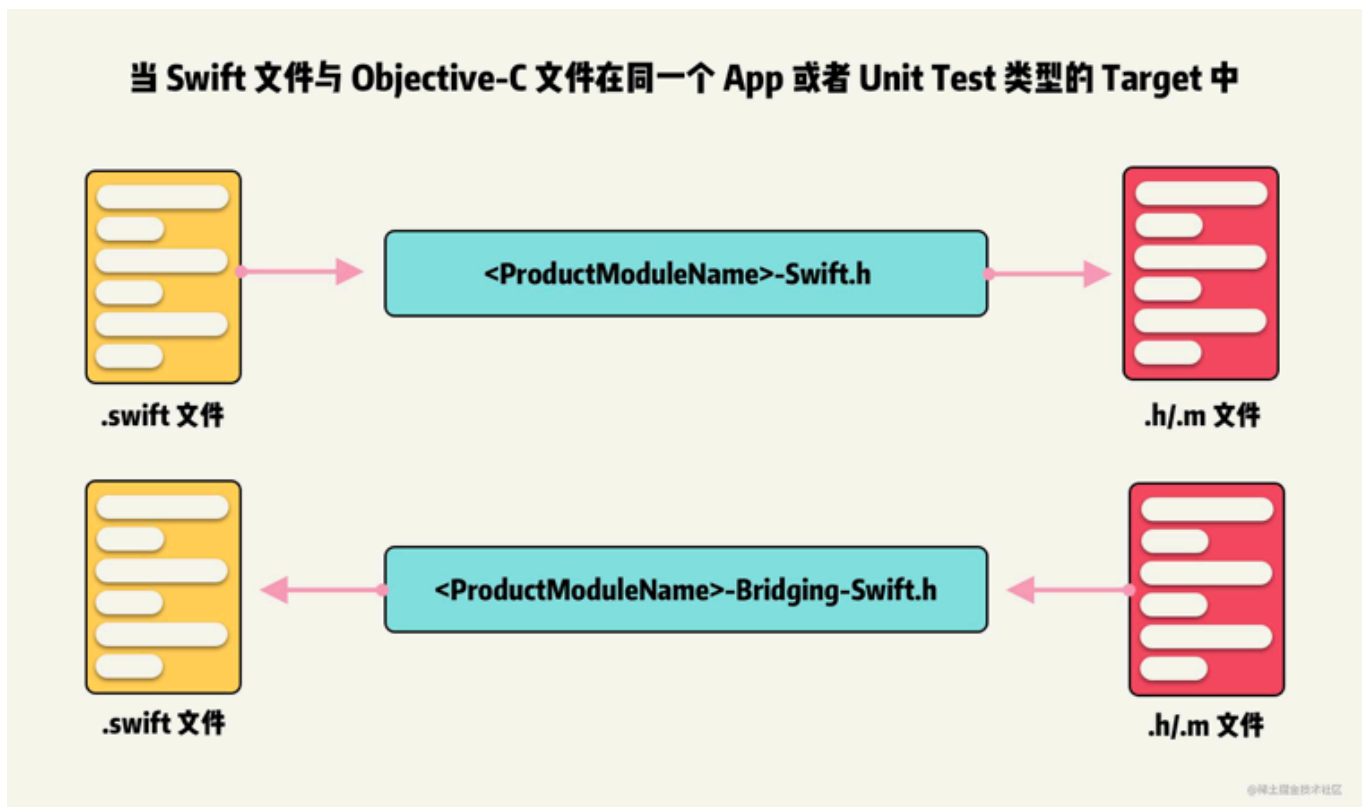
`module` 对头文件的引入和混编做了很大的优化，在引入其它组件的时候，建议以 `@import` 或者 `#import <A/A.h>` 的方式引入，其中 `#import <A/A.h>` 会最终转换为 `@import` 的方式。

现在使用pod库，以 `#import "XXX"` 和 `#import <XXX>` 的方式引入，Xcode都没有补全提示了，只有 `@import` 方式有。

如果想对 `module` 做更多的了解，建议看这篇博文：[从预编译的角度理解Swift与Objective-C及混编机制](#)，这篇博文写的非常好，内容也很长，阅读完需要一两个小时。后面的内容很多是对篇博文的部分总结和回顾，再次感谢这篇博文的作者。

pod库中swift与Objective-C的混编问题

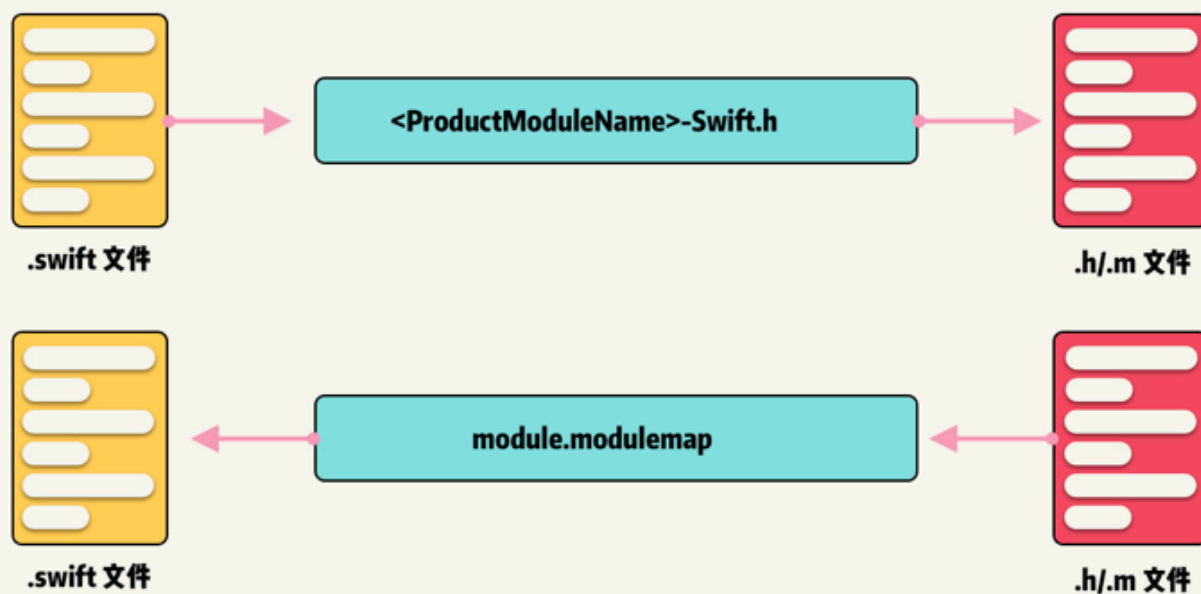
当 `Swift` 和 `Objective-C` 文件同时在一个 `App` 或者 `Unit Test` 类型的 `Target` 中，不同类型文件的 `API` 寻找机制如下：



这个和pod没有关联，是主工程的混编用法。

当 `Swift` 和 `Objective-C` 文件在不同 `Target` 中，例如不同 `Framework` 中，不同类型文件的 `API` 寻找机制如下：

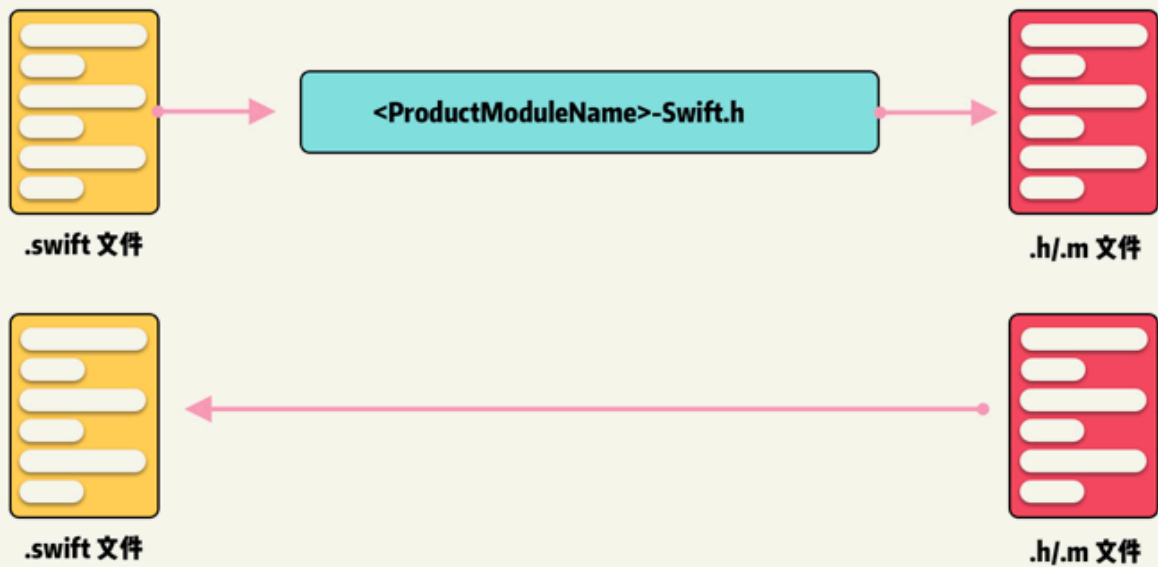
当 Swift 文件与 Objective-C 文件在不同的 Framework 中



图示的 `module.modulemap` 指的是需要 `import` 不同 `framework` 的 `module`，例如 A pod 中的 `swift` 文件需要引用 B pod 中的 `.h/.m` 文件，需要：`import B`

当 `Swift` 和 `Objective-C` 文件同时在一个 `Target` 中，例如同一个 `Framework` 中，不同类型文件的 `API` 寻找机制如下：

当 Swift 文件与 Objective-C 文件在同一个 Framework 中



这代表在同一个pod库中，不需要做任何处理，`swift` 就可以直接调用库中的 `oc` 代码。但是 `oc` 想调用库中的 `swift` 代码，则需要导入固定格式的头文件，如：`#import <DTVideo/DTVideo-Swift.h>`，其中 `DTVideo` 就是这个pod库的 `module` 名。另外还有点需要注意：

- `swift` 中必须是声明为 `public` 或者 `open` 权限的才能被同一个 `pod` 中的 `oc` 代码或者外部调用。
- 如果想在pod外部创建它某个 `swift`类的子类，那么这个 `swift`类必须声明为 `open` 权限

pod中swift使用struct的坑

`struct` 默认生成的初始化方法是 `internal` 级别的，例如：

```
public struct TestStruct {  
    let key: String  
}  
  
func test() {
```

swift 复制代码

```
TestStruct(key: "aaaa")
}
```

它可以在pod里面调用，但是在pod外部被调用则会报错： `'TestStruct' initializer is inaccessible due to 'internal' protection level`。需要手动创建它的 `public init` 方法。如：

[swift 复制代码](#)

```
public struct TestStruct {
    let key: String
    // 需要手动添加public init方法
    public init(key: String) {
        self.key= key
    }
}
```

设置pod库的 **Public Headers** 和 **Private Headers**

构建产物为 **Framework** 的情况下

- 根据 `podspec` 里的 `public_header_files` 字段的内容，将相应头文件设置为 **Public** 类型，并放在 **Headers** 中。
- 根据 `podspec` 里的 `private_header_files` 字段的内容，将相应文件设置为 **Private** 类型，并放在 **PrivateHeader** 中。
- 将其余未描述的头文件设置为 **Project** 类型，且不放入最终的产物中。

如果 `podspec` 里未标注 **Public** 和 **Private** 的时候，会将所有文件设置为 **Public** 类型，并放在 **Header** 中。

构建产物为 **Static Library** 的情况下

不论 `podspec` 里如何设置 `public_header_files` 和 `private_header_files`，相应的头文件都会被设置为 **Project** 类型。

- 在 `Pods/Headers/Public` 中会保存所有被声明为 `public_header_files` 的头文件。
- 在 `Pods/Headers/Private` 中会保存所有头文件，不论是 `public_header_files` 或者 `private_header_files` 描述到，还是那些未被描述的，这个目录下是当前组件的所有头文件全集。

关于路由

现在常见的路由方案有：`URLRoute`，`Protocol-Class`，`Target-Action` 等。个人偏爱 `URLRoute`，主要有两方面的原因：

- 第一是通用。`URLRoute` 可以多端统一，例如运营配置一条推送，点击打开某个页面，对于运营方来说，统一配置固定规则的URL就可以了，每个端都一致。H5页面要打开某个Native页面，或者外部唤起，也是统一用 `URLRoute` 就可以了，不需要区分平台做不同操作。
- 第二是没法避开。例如 `URL Scheme`，`Universal Links` 这类，还是需要使用到 `URL`。

而 `Protocol-Class`，`Target-Action` 这些方案，没法避免硬编码，只能说是 `URLRoute` 的一种补充。`URLRoute` 本质上就是约定一个各端通用的协议，在各端内部对协议进行正确的解析和逻辑处理。封装好了之后，不管是外部还是内部的调用者，不需要关心任何细节和区分平台，只需要传入协议就可以。综上所述，推荐使用 `URLRoute`。

关于脚本

组件化后会多出很多重复简单的操作，例如一个 私有pod 的新版本发布，需要的流程有：`git commit -> 打tag -> pod验证 -> pod发布`，这些都是可以通过编写脚本简化操作的，建议在组件化过程中多做这方面的工作。

后文

现在市面上的组件化方案很多，各大公司各种高大上的落地方案。我在小公司的业务间隙，抽时间写的这篇简单的避坑心得，是对自己实践的整理和归纳，希望能帮到你。

参考资料

- [从预编译的角度理解Swift与Objective-C及混编机制](#)
- [iOS 组件化 —— 路由设计思路分析](#)

by: [星的天空](#)

分类: iOS 标签: iOS CocoaPods

文章被收录于专栏:



iOS

iOS技术分享

[关注专栏](#)

安装掘金浏览器插件

多内容聚合浏览、多引擎快捷搜索、多工具便捷提效、多模式随心畅享，你想要的，这里都有！

[前往安装](#)

评论



输入评论（Enter换行，⌘ + Enter发送）

热门评论 🔥



archerq iOS开发

10天前

这里我遇到过一个问题，如果在私有pod中使用了xib加载图片，需要把这个控件拉出来再次赋值一遍图片才行。因为xib无法设置读取的bundle。只有代码才可以。类似于：`let image = DTVideoCommon.imageWith(named: "video_play_max_nor")`。请问这个怎么优化呢？

点赞 2

星的天空 Lv2 (作者)

10天前

把xib文件和图片放入同一个目录就可以直接使用的。具体我新写了一篇文章：

juejin.cn

1 回复



archerq 回复 星的天空

4天前

看过了，文章写得不错，讲的很清晰明了 🍀

“把xib文件和图片放入同一个目录就可以直接使用的。具体我... juejin.cn”

点赞 回复

全部评论 8

最新

最热

gaoronghua Lv1

2天前

都2202年了，我们部门领导还让我们适配8.0版本 😂

点赞 回复



zh_yes Lv1 iOS

5天前

想问下，完全不懂组件化的话，25k+ 工作还好找吗？

👍 点赞 1



星的天空 Lv2 (作者)

5天前

组件化更看重具体实践落地经验，只是一个知识点，也并不复杂。能否找到满意的岗位和薪资是综合各方面考量的，在大部分iOS岗位上，组件化是属于锦上添花类型的～

👍 1 回复



ice酱64794

6天前

请问，如何实现H5页面要打开某个Native页面 或者哪个路由支持？

👍 点赞 1



星的天空 Lv2 (作者)

6天前

方案有多种，三言两语讲不清楚，建议你在搜下hybrid方面的文章～

👍 点赞 回复



archerq iOS开发

10天前

这里我遇到过一个问题，如果在私有pod中使用了xib加载图片，需要把这个控件拉出来再次赋值一遍图片才行。因为xib无法设置读取的bundle。只有代码才可以。类似于：`let image = DTVideoCommon.imageWith(named: "video_play_max_nor")`。请问这个怎么优化呢？

点赞 2

星的天空 Lv2 (作者)

10天前

把xib文件和图片放入同一个目录就可以直接使用的。具体我新写了一篇文章：

juejin.cn

1 回复



archerq 回复 星的天空

4天前

看过了，文章写得不错，讲的很清晰明了 🍀

“把xib文件和图片放入同一个目录就可以直接使用的。具体我... juejin.cn”

点赞 回复

相关推荐

新生代农民工No1 18天前 iOS

iOS 组件化方案

4411 22 7

Ly梦k 3年前 iOS 架构 MVVM

iOS 一个轻量级的组件化思路

4561 38 7

Perry_6 5月前 iOS

iOS 组件化方案总结

1616 10 1

RCF 8月前 Objective-C

iOS 15 趟坑之旅

6570 29 10

村村村村村村村村村长 5月前 架构 iOS 前端

有赞组件化方案

1.1w 73 17

iOS沐橙君 3月前 iOS 架构

分享：iOS应用架构之谈谈组件化方案

5460 15 6

小顾iOSer 7月前 iOS

iOS开发App组件化之路

3222 32 7

Ginhhor大帅 2年前 iOS

iOS项目组件化历程

3561 38 2

Highmore 8月前 iOS

iOS15适配踩坑之旅

4370 6 2

阿里本地生活技术团队 3年前 APP iOS 架构

iOS 组件化 —— 路由设计思路分析

1.1w 311 8

铜板街科技 3年前 iOS

iOS组件化实践

5914 22 6

国孩 1年前 Swift

iOS组件化的那些事 - CTMediator

4916 64 7

Shuxia 1年前 iOS

iOS14 小组件 开发1

6247 22 12

被帅醒的吴宝宝 1年前 iOS

iOS 一步步带你实践组件二进制方案

7237 71 26

邦Ben 4年前 iOS GitHub CocoaPods

Masonry的使用，动画，出现问题解决等

2293 1 评论

每天写点代码 5月前 iOS Objective-C

iOS组件化

2450 8 5

猫D 4年前 APP 单元测试 iOS

iOS App组件化开发实践

1319 58 评论

taoZen 3年前 iOS Xcode CocoaPods

[iOS开发]Carthage安装和使用教程

4724 6 评论

Perry_6 1年前 Objective-C

iOS 无侵入埋点组件总结

6322 61 20

CainLuo 2年前 iOS CocoaPods

稀土掘金浏览器插件——你的一站式工作台

多内容聚合浏览、多引擎快捷搜索、多工具便捷提效、多模式随心畅享，你想要的，这里都有。

